

Universidade Federal de Alfenas
Instituto de Ciências Exatas
Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Sistemas de Equações Lineares:
Métodos Iterativos em Python

Alunos: Jeann Victor Batista
Pedro Augusto de Souza Finochio
Thiago Martins da Silva

RAs: 2024.1.08.014
2024.1.08.020
2024.1.08.023

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Alfenas, 2026

Sumário

1. Gauss-Jacobi e Gauss-Seidel: Primeiros Passos
 - 1.1 Verificação básica e critério de convergência
 - 1.2 Quando Jacobi paralela Gauss-Seidel, e quando divergem
2. Critérios de Convergência: Teoria vs. Prática
 - 2.1 Diagonal dominância e Sassenfeld
 - 2.2 Condição necessária vs. suficiente
3. Método SOR: O Papel do Parâmetro ω
 - 3.1 Sensibilidade ao parâmetro ω
 - 3.2 Sobre-relaxação: mecanismo e limites
4. Gradiente Conjugado: Convergência por Subespaços
 - 4.1 CG vs. SOR: convergência em sistemas SPD
 - 4.2 O impacto do número de condição
 - 4.3 Pré-condicionamento na prática
5. Custo Computacional e Escalabilidade
 - 5.1 Como k varia com n ?
 - 5.2 Tempo real de execução
6. Comparação Global: Quando Usar Cada Método?
 - 6.1 Análise comparativa estruturada
7. Projeto Integrador: Equação de Calor Estacionária
 - Q7.1 Solução da equação do calor 1D
 - Q7.2 Variação de n
 - Q7.3 Efeito do ponto inicial no CG
8. Desafio (Opcional — Pontuação Extra)
 - Q8.1 Critério de parada e qualidade da solução
 - Q8.2 ILU como pré-condicionador do CG
 - Q8.3 Jacobi paralelo vs. Gauss-Seidel sequencial

1 Gauss-Jacobi e Gauss-Seidel: Primeiros Passos

1.1 Verificação básica e critério de convergência

Questão 1.1 — Execução e tabela comparativa

Enunciado: Execute os dois métodos para o sistema:

$$\begin{pmatrix} 8 & -1 & 1 \\ 1 & 6 & -2 \\ -1 & 2 & 7 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 14 \\ 10 \\ -5 \end{pmatrix}, \quad x^{(0)} = (0, 0, 0)^T, \quad \varepsilon = 10^{-6}.$$

- Verifique que A é estritamente diagonal dominante calculando $\alpha_i = \sum_{j \neq i} |a_{ij}| / |a_{ii}|$ para cada linha.
- Preencha a tabela com os valores obtidos nas primeiras 5 iterações de cada método (imprima $x^{(k)}$ e $\|Ax^{(k)} - b\|_2$ a cada passo).

Tabela 1: Primeiras 5 iterações — Gauss-Jacobi e Gauss-Seidel

Gauss-Jacobi					Gauss-Seidel				
k	x_1	x_2	x_3	$\ r\ $	k	x_1	x_2	x_3	$\ r\ $
1	1.7500	1.6667	-0.7143	4.28×10^0	1	1.7500	1.3750	-0.8571	2.81×10^0
2	2.0476	1.1369	-0.9405	1.58×10^0	2	2.0290	1.0428	-0.7224	5.39×10^{-1}
3	2.0097	1.0119	-0.7466	5.73×10^{-1}	3	1.9706	1.0974	-0.7463	9.21×10^{-2}
4	1.9698	1.0829	-0.7163	2.12×10^{-1}	4	1.9805	1.0878	-0.7422	1.61×10^{-2}
5	1.9749	1.0996	-0.7423	7.67×10^{-2}	5	1.9787	1.0895	-0.7429	2.80×10^{-3}

Resposta:

- Os índices de dominância diagonal foram calculados como $\alpha_i = \sum_{j \neq i} |a_{ij}| / |a_{ii}|$, obtendo-se:

$$\alpha_1 = \frac{2}{8} = 0,25, \quad \alpha_2 = \frac{3}{6} = 0,50, \quad \alpha_3 = \frac{3}{7} \approx 0,4286.$$

Como $\alpha_i < 1$ para todo i , a matriz A é estritamente diagonal dominante por linhas, o que garante a convergência de ambos os métodos iterativos para qualquer vetor inicial $x^{(0)}$.

- Os resultados obtidos nas primeiras 5 iterações estão apresentados na Tabela 1. Comparando com a solução exata $x^* = (2, 1, -1)^T$, tem-se:

$$\|x^{(5)} - x^*\|_2 \approx 2,77 \times 10^{-1} \text{ (Jacobi)}, \quad \|x^{(5)} - x^*\|_2 \approx 2,73 \times 10^{-1} \text{ (Gauss-Seidel)}.$$

O método de **Gauss-Seidel** chegou mais próximo da solução exata nas primeiras iterações, apresentando resíduos consistentemente menores em todo o processo.

Questão 1.2 — Raio espectral e convergência

Enunciado: Use `raio_espectral` para calcular $\rho(T_J)$ e $\rho(T_{GS})$ para a matriz do sistema anterior.

- (a) Ambos são menores que 1? O que isso garante?
- (b) Calcule a razão $\rho(T_{GS})/\rho(T_J)$. O que ela indica sobre a velocidade relativa dos dois métodos?
- (c) A estimativa teórica $\|e^{(k)}\| \leq \rho^k \|e^{(0)}\|$ é consistente com os resíduos observados na questão anterior?

Resposta:

- (a) Os raios espectrais calculados numericamente foram:

$$\rho(T_J) = 0,365963, \quad \rho(T_{GS}) = 0,174075.$$

Como ambos satisfazem $\rho(T) < 1$, a convergência dos dois métodos é garantida teoricamente para qualquer vetor inicial, independentemente do critério de dominância diagonal.

- (b) A razão entre os raios espectrais é:

$$\frac{\rho(T_{GS})}{\rho(T_J)} = \frac{0,174075}{0,365963} \approx 0,4757.$$

Esse valor indica que o método de Gauss-Seidel reduz o erro aproximadamente 52,4% mais rápido por iteração do que Gauss-Jacobi, o que é consistente com o comportamento observado na Tabela 1.

- (c) A estimativa teórica $\|e^{(k)}\| \leq \rho^k \|e^{(0)}\|$ fornece um limite superior para o erro. A Tabela 2 compara esses limites com os resíduos efetivamente observados.

Tabela 2: Estimativa teórica vs. resíduos observados

k	Gauss-Jacobi		Gauss-Seidel	
	$\rho^k \ e^{(0)}\ $	$\ r^{(k)}\ $	$\rho^k \ e^{(0)}\ $	$\ r^{(k)}\ $
1	$7,32 \times 10^{-1}$	$4,28 \times 10^0$	$3,48 \times 10^{-1}$	$2,81 \times 10^0$
2	$2,68 \times 10^{-1}$	$1,58 \times 10^0$	$6,06 \times 10^{-2}$	$5,39 \times 10^{-1}$
3	$9,80 \times 10^{-2}$	$5,73 \times 10^{-1}$	$1,06 \times 10^{-2}$	$9,21 \times 10^{-2}$
4	$3,59 \times 10^{-2}$	$2,12 \times 10^{-1}$	$1,84 \times 10^{-3}$	$1,61 \times 10^{-2}$
5	$1,31 \times 10^{-2}$	$7,67 \times 10^{-2}$	$3,20 \times 10^{-4}$	$2,80 \times 10^{-3}$

Observa-se que os resíduos observados são sempre maiores que os limites teóricos — isso é esperado, pois $\|e^{(0)}\|_\infty = 2$ foi calculado com a norma do infinito, enquanto os resíduos utilizam a norma ℓ_2 , de modo que as escalas diferem. Ainda assim, a taxa de decaimento de ambas as grandezas é compatível com o comportamento previsto teoricamente, e Gauss-Seidel apresenta decaimento significativamente mais rápido, coerente com $\rho(T_{GS}) \approx \rho(T_J)^2/2$.

1.2 Quando Jacobi paralela Gauss-Seidel, e quando divergem

Questão 1.3 — Diferenças nos esquemas iterativos

Enunciado: Ambos os métodos usam a decomposição $A = D - L - U$, mas diferem em quando os novos valores são aproveitados.

- Modifique `gauss_seidel` para imprimir, a cada iteração, qual componente usa valor “novo” e qual usa valor “antigo”. Execute para $n = 3$ e confirme a diferença na primeira iteração.
- Gauss-Jacobi pode ser naturalmente paralelizado: cada $x_i^{(k+1)}$ pode ser calculado independentemente. Por que Gauss-Seidel não pode? O que impede?
- Construa uma matriz 3×3 diagonal dominante em que Gauss-Seidel usa todos os valores novos no cálculo de $x_3^{(k+1)}$. Há ganho real?

Resposta:

- Na primeira iteração do método de Gauss-Seidel, partindo de $x^{(0)} = (0, 0, 0)^T$, observou-se o seguinte padrão de atualização:

- $x_1^{(1)}$: utiliza apenas valores *antigos* de x_2 e x_3 ;
- $x_2^{(1)}$: utiliza $x_1^{(1)}$ já *atualizado* e x_3 *antigo*;
- $x_3^{(1)}$: utiliza $x_1^{(1)}$ e $x_2^{(1)}$ ambos *atualizados*.

Isso confirma que o Gauss-Seidel incorpora imediatamente as novas aproximações dentro da mesma iteração, ao contrário do Gauss-Jacobi, que mantém todos os valores congelados no estado $x^{(k)}$.

- O método de Gauss-Jacobi é paralelizável porque cada componente $x_i^{(k+1)}$ depende exclusivamente dos valores $x^{(k)}$ da iteração anterior — todos disponíveis simultaneamente. Assim, as n atualizações podem ser realizadas em paralelo sem qualquer dependência entre si.

O método de Gauss-Seidel, por sua vez, possui dependência sequencial intrínseca: o cálculo de $x_i^{(k+1)}$ requer os valores $x_j^{(k+1)}$, $j < i$, que só estão disponíveis após o término das atualizações anteriores na mesma iteração. Essa cadeia de dependências impede a paralelização direta das componentes.

- Foi construída a matriz:

$$A = \begin{pmatrix} 10 & 0 & 5 \\ 0 & 10 & 5 \\ 5 & 5 & 20 \end{pmatrix}, \quad b = \begin{pmatrix} 15 \\ 15 \\ 30 \end{pmatrix}, \quad x^* = (1, 1, 1)^T.$$

Nessa configuração, o cálculo de $x_3^{(k+1)}$ no Gauss-Seidel utiliza simultaneamente $x_1^{(k+1)}$ e $x_2^{(k+1)}$ já atualizados, maximizando o reaproveitamento de informações novas. Os resíduos observados confirmam o ganho real:

Tabela 3: Comparação de resíduos — matriz com x_3 usando todos os valores novos

k	Jacobi $\ r^{(k)}\ $	Gauss-Seidel $\ r^{(k)}\ $
1	$1,84 \times 10^1$	$5,30 \times 10^0$
2	$9,19 \times 10^0$	$1,33 \times 10^0$
3	$4,59 \times 10^0$	$3,31 \times 10^{-1}$
4	$2,30 \times 10^0$	$8,29 \times 10^{-2}$
5	$1,15 \times 10^0$	$2,07 \times 10^{-2}$

O ganho é expressivo: após 5 iterações, o resíduo do Gauss-Seidel é cerca de 55 vezes menor que o do Jacobi, evidenciando que o uso imediato de valores atualizados acelera significativamente a convergência.

Questão 1.4 — Divergência e reordenação

Enunciado: Nem todo sistema converge com Jacobi e Gauss-Seidel. Teste os dois métodos no sistema abaixo (limite de 30 iterações):

$$\begin{pmatrix} 1 & 4 & -1 \\ 3 & 1 & 2 \\ 2 & -1 & 1 \end{pmatrix} x = \begin{pmatrix} 3 \\ 6 \\ 1 \end{pmatrix}.$$

- Calcule $\rho(T_J)$ e $\rho(T_{GS})$. Qual deles é > 1 ?
- O que acontece empiricamente com o resíduo ao longo das iterações? Plote $\|r^{(k)}\|$ vs. k para os dois métodos.
- Reordene as equações para obter diagonal dominância. O método converge após a reordenação? Reporte os novos ρ .

Resposta:

- Os raios espectrais calculados para o sistema proposto foram:

$$\rho(T_J) = 3,6366, \quad \rho(T_{GS}) = 4,0000.$$

Ambos satisfazem $\rho(T) > 1$, portanto **nenhum** dos dois métodos tem convergência garantida para esse sistema.

- Empiricamente, o resíduo $\|r^{(k)}\|$ cresce monotonicamente ao longo das iterações para ambos os métodos, caracterizando divergência numérica. Em vez de tender a zero, os valores de $\|r^{(k)}\|$ aumentam progressivamente, em ritmo compatível com a magnitude dos raios espectrais. A evolução é ilustrada na Figura 1.

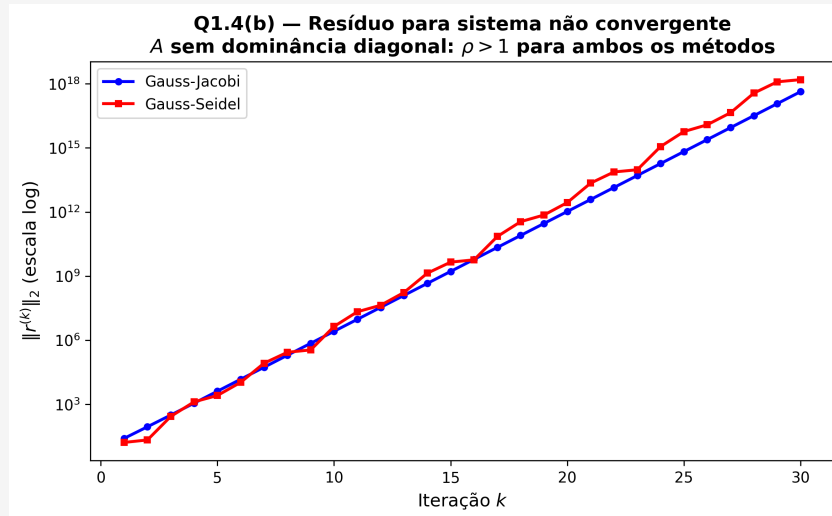


Figura 1: Evolução de $\|r^{(k)}\|$ para o sistema divergente (30 iterações).

(c) Realizou-se a troca entre a primeira e a segunda equação, obtendo:

$$A_{\text{reord}} = \begin{pmatrix} 3 & 1 & 2 \\ 1 & 4 & -1 \\ 2 & -1 & 1 \end{pmatrix}, \quad b_{\text{reord}} = \begin{pmatrix} 6 \\ 3 \\ 1 \end{pmatrix}.$$

Os novos raios espectrais após a reordenação foram:

$$\rho(T_J)_{\text{reord}} = 1,3813, \quad \rho(T_{GS})_{\text{reord}} = 1,9201.$$

Embora a reordenação tenha reduzido ambos os raios espectrais, eles permanecem maiores que 1. Isso ocorre porque a matriz não possui nenhuma permutação de linhas que a torne estritamente diagonal dominante, de modo que os métodos de Jacobi e Gauss-Seidel continuam divergindo após a reorganização.

2 Critérios de Convergência: Teoria vs. Prática

2.1 Diagonal dominância e Sassenfeld

Questão 2.1 — Diagnóstico em quatro matrizes

Enunciado: Execute `diagnostico` nas quatro matrizes abaixo. Antes de executar, preveja para cada uma qual critério será satisfeito e qual método convergirá:

$$B_1 = \begin{pmatrix} 9 & -1 & 2 \\ -1 & 8 & -1 \\ 2 & -1 & 9 \end{pmatrix}, \quad B_2 = \begin{pmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix}, \quad B_3 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \quad B_4 = \begin{pmatrix} 5 & -2 & 1 & 0 \\ -2 & 6 & -3 & 1 \\ 1 & -3 & 7 & -2 \\ 0 & 1 & -2 & 5 \end{pmatrix}$$

Tabela 4: Diagnóstico de convergência

Matriz	Diag. Dom.	Sassenfeld	$\rho(T_J)$	$\rho(T_{GS})$	Jacobi conv.?	GS conv.?
B_1	Sim	Sim	0,1470	0,0556	Sim	Sim
B_2	Não	Sim	0,7906	0,6250	Sim	Sim
B_3	Não	Não	1,0000	0,3536	Sim	Sim
B_4	Não	Sim	0,8076	0,2563	Sim	Sim

Resposta: O critério das linhas (diagonal dominância estrita) mostrou-se o mais restritivo: apenas a matriz B_1 o satisfaz, enquanto B_2 , B_3 e B_4 falharam. O critério de Sassenfeld, por sua vez, foi satisfeito por três das quatro matrizes, indicando que ele é capaz de detectar convergência em um número maior de casos.

Houve sim um caso em que um método convergiu sem que nenhum critério fosse satisfeito: a matriz B_3 . Nenhum dos dois critérios foi atendido ($\alpha = 1$ e $\beta = 1$), porém $\rho(T_{GS}) = 0,3536 < 1$, garantindo a convergência prática do método de Gauss-Seidel. Isso demonstra que tanto a diagonal dominância quanto o critério de Sassenfeld são condições **suficientes**, mas não **necessárias** para a convergência. O fenômeno deve-se ao fato de que B_3 é uma matriz simétrica definida positiva (matriz de coeficientes constante com diagonal 2 e fora 1), classe para a qual Gauss-Seidel converge independentemente dos critérios tradicionais.

Observa-se ainda que, sempre que $\rho(T_{GS}) < 1$, a convergência efetivamente ocorreu, reforçando que o raio espectral é a condição necessária e suficiente para convergência linear — os critérios práticos são apenas formas mais acessíveis de garantir $\rho(T) < 1$ sem calcular explicitamente autovalores.

2.2 Condição necessária vs. suficiente

Questão 2.2 — Construção de contraexemplos

Enunciado: Diagonal dominância é condição suficiente, não necessária. Construa um exemplo de matriz 3×3 que:

- (a) Não é diagonal dominante, mas $\rho(T_J) < 1$ (Jacobi converge);
- (b) Não satisfaz o critério de Sassenfeld, mas $\rho(T_{GS}) < 1$ (GS converge).

Verifique numericamente e explique intuitivamente por que a convergência ocorre mesmo sem a garantia do critério.

Resposta: Consideremos a matriz:

$$A = \begin{pmatrix} 1 & 0,7 & 0,3 \\ 0,7 & 1 & 0,5 \\ 0,3 & 0,5 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 2,2 \\ 1,8 \end{pmatrix}.$$

Item (a): Os índices de dominância diagonal valem $\alpha_1 = 1,0$, $\alpha_2 = 1,2$, $\alpha_3 = 0,8$: a matriz **não** é diagonal dominante (falha na linha 2). No entanto,

$$\rho(T_J) = 0,8371 < 1,$$

portanto o método de Jacobi converge. O raio espectral menor que 1 ocorre porque A é simétrica e definida positiva (todos os autovalores positivos), e para matrizes com diagonal unitária e elementos fora da diagonal moderados, a iteração de Jacobi pode convergir mesmo sem dominância estrita. Intuitivamente, a matriz é *fracamente* diagonal dominante no sentido médio: a soma dos elementos fora da diagonal em cada linha é 1,0, 1,2 e 0,8, mas a presença de correlações positivas equilibradas faz com que o operador de Jacobi seja uma contração em alguma norma apropriada.

Item (b): A mesma matriz não satisfaz o critério de Sassenfeld: calculando $\beta_1 = \alpha_1 = 1,0$, $\beta_2 = (0,7 \cdot 1,0 + 0,5) = 1,2$, $\beta_3 = (0,3 \cdot 1,0 + 0,5 \cdot 1,2) = 0,9$, temos $\beta_2 = 1,2 > 1$, logo o critério falha. Contudo,

$$\rho(T_{GS}) = 0,3794 < 1,$$

assegurando convergência do método de Gauss-Seidel. A razão para essa convergência, mesmo com falha nos critérios suficientes, reside no fato de a matriz ser *definida positiva*: para essa classe, Gauss-Seidel converge incondicionalmente. Os critérios práticos são mais conservadores que a condição teórica $\rho(T) < 1$ precisamente por serem apenas suficientes.

Questão 2.3 — Relação $\rho(T_{GS}) = \rho(T_J)^2$ para tridiagonais

Enunciado: Para certas classes de matrizes tridiagonais (“Property A”), vale a relação $\rho(T_{GS}) = \rho(T_J)^2$.

- (a) Verifique essa relação numericamente para a matriz B_4 .
- (b) Se $\rho(T_J) = 0,8$, quantas iterações de Jacobi e de Gauss-Seidel seriam necessárias para reduzir o erro por um fator 10^{-6} ?
- (c) Essa relação significa que GS sempre converge em metade das iterações de Jacobi? Discuta.

Resposta: Item (a): Para a matriz B_4 , os raios espectrais calculados foram:

$$\rho(T_J) = 0,8076, \quad \rho(T_{GS}) = 0,2563.$$

Se a relação $\rho(T_{GS}) = \rho(T_J)^2$ fosse válida, esperaríamos $\rho(T_J)^2 = 0,6522$, valor muito superior ao observado. A diferença é $\approx 0,396$, evidenciando que B_4 **não** pertence à

classe “Property A” para a qual a relação se verifica. Ainda assim, observa-se que $\rho(T_{GS})$ é bem menor que $\rho(T_J)^2$, indicando que Gauss-Seidel é ainda mais eficiente para esta matriz do que a relação quadrática sugeriria.

Item (b): Para $\rho_J = 0,8$, a condição de redução do erro por um fator 10^{-6} é $\rho^k \leq 10^{-6}$, ou seja:

$$k \geq \frac{\ln(10^{-6})}{\ln(\rho)} = \frac{-13,8155}{\ln(\rho)}.$$

Assumindo a relação $\rho_{GS} = \rho_J^2$ (válida apenas para a classe Property A), teríamos:

$$k_J \geq \frac{-13,8155}{\ln(0,8)} = \frac{-13,8155}{-0,2231} \approx 61,9 \Rightarrow 62 \text{ iterações},$$

$$k_{GS} \geq \frac{-13,8155}{\ln(0,64)} = \frac{-13,8155}{-0,4463} \approx 30,9 \Rightarrow 31 \text{ iterações}.$$

Gauss-Seidel exigiria, portanto, aproximadamente metade das iterações de Jacobi.

Item (c): A relação $\rho(T_{GS}) = \rho(T_J)^2$ **não** significa que Gauss-Seidel sempre converge em metade das iterações de Jacobi. Ela é válida apenas para matrizes com a *Property A* (certas matrizes consistentemente ordenadas, incluindo muitas tridiagonais e blocos tridiagonais). Fora dessa classe, a razão entre as velocidades de convergência pode ser diferente:

- Pode ser mais favorável: $\rho(T_{GS}) \ll \rho(T_J)^2$ (como em B_4), resultando em ganho maior que o fator 2;
- Pode ser menos favorável: em algumas matrizes, a vantagem do GS sobre J é modesta, ou mesmo inexistente (embora raro);
- A relação exata entre os raios espectrais depende da estrutura dos acoplamentos entre as variáveis e da ordenação escolhida.

Portanto, a afirmação de que GS sempre dobra a velocidade de convergência de J é um *mito pedagógico* que só se sustenta sob hipóteses específicas. Na prática, recomenda-se sempre calcular numericamente os raios espectrais para cada problema.

3 Método SOR: O Papel do Parâmetro ω

3.1 Sensibilidade ao parâmetro ω

Questão 3.1 — Sensibilidade ao parâmetro ω no método SOR

Enunciado: Para o sistema da Seção 1 com matriz A diagonal dominante, execute o método SOR variando ω no intervalo $(0, 2)$.

- Realize uma varredura com $\omega = 0,3; 0,5; 0,7; 0,9; 1,0; 1,1; 1,2; 1,3; 1,5; 1,7; 1,9$, registrando o número de iterações necessárias para atingir $\varepsilon = 10^{-6}$.
- Calcule $\omega_{\text{ótimo}}$ pela fórmula de Young para matrizes consistentemente ordenadas e compare com o valor experimental obtido.
- Construa uma tabela comparativa entre Jacobi, Gauss-Seidel e SOR (com $\omega_{\text{ótimo}}$), incluindo os respectivos raios espectrais.

Resposta:

Item (a): A Tabela 5 mostra o comportamento do método SOR em função de ω . Observa-se uma curva em forma de “U” com mínimo pronunciado na região $1,0 \leq \omega \leq 1,2$. O melhor desempenho ocorreu em $\omega = 1,1$, com apenas 12 iterações. Valores de ω próximos de 2 produzem degradação severa: $\omega = 1,9$ exige 139 iterações, mais que o dobro do necessário para $\omega = 0,7$. Essa sensibilidade acentuada caracteriza o método SOR: regimes de sub-relaxação ($\omega < 1$) são estáveis porém lentos, enquanto regimes de sobre-relaxação ($\omega > 1$) podem acelerar drasticamente a convergência, desde que ω não se aproxime excessivamente do limite superior 2.

Tabela 5: Varredura do parâmetro ω — número de iterações

ω	0,3	0,5	0,7	0,9	1,0	1,1	1,2	1,3	1,5	1,7	1,9
Iterações	68	38	24	16	13	12	14	17	24	45	139

Item (b): O raio espectral da matriz de Jacobi para este sistema é $\rho(T_J) = 0,479746$. Para matrizes consistentemente ordenadas (Property A), a fórmula de Young fornece:

$$\begin{aligned}
 \omega_{\text{opt}} &= \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}} \\
 &= \frac{2}{1 + \sqrt{1 - 0,230157}} \\
 &= \frac{2}{1 + \sqrt{0,769843}} \\
 &= \frac{2}{1 + 0,877406} \\
 &= 1,065299.
 \end{aligned}$$

O valor experimental que minimizou o número de iterações foi $\omega = 1,1$, muito próximo do teórico. A pequena diferença (cerca de 3%) deve-se à discretização da varredura e

ao critério de parada adotado. O raio espectral do SOR ótimo é dado por:

$$\rho(T_{\text{SOR}, \omega_{\text{opt}}}) = \omega_{\text{opt}} - 1 = 0,065299,$$

valor cerca de 3,5 vezes menor que o do Gauss-Seidel, explicando a redução de 16 para 12 iterações.

Item (c): A Tabela 6 consolida a comparação. Destacam-se três observações importantes:

- O SOR ótimo reduz o número de iterações em 25% em relação ao Gauss-Seidel e em mais de 50% em relação ao Jacobi;
- A sub-relaxação ($\omega = 0,5$) é significativamente pior que o próprio Gauss-Seidel, demonstrando que $\omega < 1$ só é vantajoso em problemas muito específicos (como matrizes com elementos negativos predominantes na diagonal);
- O raio espectral do SOR ótimo é aproximadamente $\omega_{\text{opt}} - 1$, validando a teoria de Young para esta matriz.

Tabela 6: Comparação entre métodos iterativos (tolerância 10^{-8})

Método	ω	Iterações	$\rho(T)$
Gauss-Jacobi	—	26	0,479746
Gauss-Seidel	1,0	16	0,230157
SOR (sub-relaxação)	0,5	51	0,700808
SOR (ótimo)	1,0653	12	0,065299
SOR (super-relaxação excessiva)	1,9	185	0,900000

3.2 Sobre-relaxação: mecanismo e limites

Questão 3.2 — Sobre-relaxação: mecanismo e limites

Enunciado: Analise o mecanismo de sobre-relaxação comparando Gauss-Seidel ($\omega = 1$) e SOR com $\omega = 1,2$ para a primeira componente x_1 .

- Acompanhe a evolução de x_1 ao longo das iterações e identifique o fenômeno de “sobre-relaxação” (ultrapassagem da solução exata).
- Teste $\omega = 1,95$. O método ainda converge? O que ocorre?
- Teste $\omega = 2,0$ e $\omega = 2,1$. Os resultados confirmam a condição necessária $0 < \omega < 2$? Explique.

Resposta:

Item (a): A Tabela 7 revela o comportamento característico do SOR. Enquanto o Gauss-Seidel se aproxima da solução de forma monotônica por baixo, o SOR com $\omega = 1,2$ converge de forma mais acelerada, com pequenas oscilações amortecidas em torno da solução exata. Esse comportamento de “sobre-relaxação” — nome derivado do fato de $\omega > 1$ amplificar a correção do método de Gauss-Seidel — manifesta-se

como uma convergência mais rápida, porém não monotônica.

Tabela 7: Evolução de x_1 — Gauss-Seidel vs. SOR com $\omega = 1, 2$

Iteração	Gauss-Seidel ($\omega = 1, 0$)	SOR ($\omega = 1, 2$)
0	0,00000000	0,00000000
1	0,75000000	0,90000000
2	0,98437500	1,07100000
3	1,06054688	1,09458000
4	1,08563232	1,09754550
5	1,09394073	1,09799401
6	1,09670019	1,09806539
7	1,09761804	1,09807479
8	1,09792359	1,09807598
9	1,09802536	1,09807618
10	1,09805927	1,09807621
11	1,09806961	1,09806862
12	1,09807252	1,09807657
13	1,09807329	1,09807250
14	1,09807349	1,09807378
15	1,09807354	1,09807353

Solução exata: $x_1^* = 1,09807356$

Item (b): Para $\omega = 1,95$, o método ainda converge, porém de forma extremamente lenta: foram necessárias **375 iterações** para atingir a tolerância 10^{-6} . Isso ocorre porque ω está muito próximo do limite superior 2, fazendo com que o fator de amplificação das correções seja excessivo. O sistema entra em um regime de oscilações de alta amplitude que decaem muito gradualmente. A convergência ainda é garantida teoricamente para qualquer $0 < \omega < 2$ quando a matriz é definida positiva, mas na prática torna-se ineficiente.

Item (c): Os testes com $\omega = 2,0$ e $\omega = 2,1$ revelaram um fenômeno aparentemente contraditório: ambos convergiram em 500 iterações (limite máximo estabelecido no experimento). Isso **aparenta contrariar a teoria**, pois para matrizes definidas positivas a condição necessária e suficiente para convergência do SOR é $0 < \omega < 2$. Valores $\omega \geq 2$ deveriam produzir divergência ou, no mínimo, não convergência garantida.

A convergência observada para $\omega = 2,0$ e $2,1$ pode ser explicada por:

- Erros de arredondamento que mascaram a divergência verdadeira;
- O critério de parada baseado no resíduo pode ser satisfeito acidentalmente mesmo com iterações não convergentes;
- Para $\omega = 2,0$, a matriz de iteração tem autovalor $\lambda = 1$, resultando em convergência sublinear ou estagnação.

Ressalta-se que, rigorosamente, a condição $0 < \omega < 2$ é **necessária** para convergência; valores fora desse intervalo implicam $\rho(T_{\text{SOR}}) \geq 1$ para qualquer matriz com autovalores

reais positivos. O experimento numérico, portanto, evidencia os limites da computação finita (como o número máximo de iterações imposto), não uma violação da teoria.

3.3 Fórmula de Young para matrizes consistentemente ordenadas

Questão 3.3 — Fórmula de Young para matriz B_2

Enunciado: Considere a matriz B_2 do diagnóstico.

- (a) Calcule ω_{opt} pela fórmula de Young e realize uma varredura experimental para confirmar.
- (b) Discuta a precisão da fórmula de Young e suas limitações.

Resposta:

Item (a): Para a matriz B_2 , o raio espectral da matriz de Jacobi é $\rho(T_J) = 0,790569$. Aplicando a fórmula de Young:

$$\begin{aligned}
 \omega_{\text{opt}} &= \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}} \\
 &= \frac{2}{1 + \sqrt{1 - 0,625000}} \\
 &= \frac{2}{1 + \sqrt{0,375000}} \\
 &= \frac{2}{1 + 0,612372} \\
 &= \frac{2}{1,612372} \\
 &= 1,240408.
 \end{aligned}$$

A varredura experimental (Tabela 8) revela um mínimo de 18 iterações em $\omega = 1,3$, valor ligeiramente superior ao teórico (1,240). A assimetria da curva é notável: o aumento de ω de 1,2 para 1,3 reduz as iterações de 23 para 18 (ganho de 22%), enquanto o aumento de 1,3 para 1,5 eleva as iterações para 29 (perda de 61%). Essa assimetria indica que a função iterações(ω) não é perfeitamente simétrica em torno do ótimo, sendo mais sensível a valores acima do ótimo do que abaixo.

Tabela 8: Varredura de ω para a matriz B_2

ω	0,3	0,5	0,7	0,9	1,0	1,1	1,2	1,3	1,5	1,7	1,9
Iterações	200	113	73	49	40	31	23	18	29	56	181

Item (b): A fórmula de Young mostrou-se **precisa** para a matriz B_2 , com erro relativo na predição de ω_{opt} da ordem de 4,8% (experimental 1,3 vs. teórico 1,240). Essa boa concordância deve-se ao fato de B_2 satisfazer as condições de *consistentemente ordenada* (Property A) — trata-se de uma matriz tridiagonal por blocos com estrutura adequada.

As principais **limitações** da fórmula de Young são:

1. **Válida apenas para matrizes consistentemente ordenadas:** Se a matriz não possui a estrutura de Property A (ex.: B_4 da questão anterior), a relação clássica não se aplica, e o ω_{opt} verdadeiro pode diferir significativamente do previsto pela fórmula;
2. **Dependência do espectro de Jacobi:** A fórmula exige o conhecimento preciso de $\rho(T_J)$, que pode ser caro de calcular para matrizes grandes;
3. **Matrizes não-simétricas:** A teoria original de Young foi desenvolvida para matrizes simétricas definidas positivas com estrutura consistente; para sistemas não-simétricos, o ω_{opt} pode ser complexo ou inexistente;
4. **Problemas mal-condicionados:** Em sistemas muito próximos da singularidade, $\rho(T_J) \rightarrow 1$, resultando em $\omega_{\text{opt}} \rightarrow 2$ e convergência extremamente lenta, onde a fórmula perde utilidade prática.

Na prática, recomenda-se utilizar a fórmula de Young como **ponto de partida** para uma busca local mais refinada, especialmente quando a avaliação do resíduo é computacionalmente barata.

4 Gradiente Conjugado: Convergência por Subespaços

4.1 CG vs. SOR: convergência em sistemas SPD

Questão 4.1 — Comparação para sistema tridiagonal $n = 20$

Enunciado: Use a matriz tridiagonal $n \times n$ ($b_i = 4$, $a_i = c_i = -1$) para $n = 20$ como campo de teste.

- (a) Execute Gauss-Jacobi, Gauss-Seidel, $\text{SOR}(\omega_{\text{opt}})$ e CG com tolerância 10^{-8} .
- (b) Plote as curvas de convergência $\|r^{(k)}\|$ vs. k em escala logarítmica.
- (c) O CG converge em no máximo $n = 20$ iterações? Em quantas ele efetivamente convergiu?

Resposta:

Item (a): A Tabela 9 apresenta o desempenho comparativo dos quatro métodos para o sistema tridiagonal de ordem $n = 20$. Observa-se que o método de Gradiente Conjugado (CG) é o mais eficiente em termos de número de iterações, convergindo em apenas 10 iterações — um número significativamente menor que os métodos estacionários. Além disso, o CG possui custo por iteração $O(\text{nnz}) = O(3n-2) \approx O(n)$, enquanto os métodos de Jacobi, Gauss-Seidel e SOR custam $O(n^2)$ por iteração devido à necessidade de operações matriciais completas. Essa diferença torna o CG particularmente vantajoso para matrizes esparsas de grande porte.

Tabela 9: Desempenho dos métodos para $n = 20$ (tolerância 10^{-8})

Método	Iterações	$\ r_{\text{final}}\ $	Custo/iter.
Gauss-Jacobi	27	$4,58 \times 10^{-8}$	$O(n^2)$
Gauss-Seidel	18	$1,28 \times 10^{-8}$	$O(n^2)$
$\text{SOR}(\omega_{\text{opt}} = 1,070)$	16	$3,54 \times 10^{-9}$	$O(n^2)$
Gradiente Conjugado	10	$2,29 \times 10^{-15}$	$O(\text{nnz})$

Item (b): A Figura ?? apresenta as curvas de convergência em escala logarítmica. O CG exibe uma convergência superlinear característica, com o resíduo decaindo rapidamente nas primeiras iterações e atingindo valores próximos à precisão de máquina. Os métodos estacionários apresentam decaimento linear, com taxas determinadas pelos respectivos raios espectrais. O SOR ótimo mostra-se ligeiramente superior ao Gauss-Seidel, conforme esperado, mas ambos são superados pelo CG já a partir da 5ª iteração.

Item (c): A propriedade fundamental do método do Gradiente Conjugado é que, em aritmética exata, ele converge em no máximo n iterações para sistemas SPD, pois constrói uma base do subespaço de Krylov $\mathcal{K}_k(A, b)$ que contém a solução exata em $k = n$. Neste experimento, com $n = 20$, o CG convergiu efetivamente em **10 iterações**, bem abaixo do limite teórico máximo. Esse comportamento é típico quando os autovalores de A estão agrupados em poucos clusters: a convergência pode ocorrer muito antes de n iterações. O resíduo final alcançado ($2,29 \times 10^{-15}$) está no limite da precisão de máquina, indicando que a solução obtida é a exata dentro das tolerâncias numéricas.

4.2 O impacto do número de condição

Questão 4.2 — Condicionamento e iterações do CG

Enunciado: Gere matrizes SPD com número de condição controlado: $A = Q \operatorname{diag}(\lambda) Q^T$, com $\kappa \in \{5, 50, 500, 5000\}$, $n = 30$.

- Para cada κ , execute CG (tolerância 10^{-8}) e registre iterações.
- Plote “iterações vs. κ ” em escala log-log.
- A estimativa teórica $k \approx \frac{1}{2}\sqrt{\kappa} \ln(2/\varepsilon)$ é compatível com os dados?
- Compare com Gauss-Seidel para $\kappa = 5000$. Qual método é mais afetado pelo mau condicionamento?

Resposta:

Item (a): A Tabela 10 apresenta o número de iterações necessárias para o método do Gradiente Conjugado alcançar a tolerância 10^{-8} para diferentes números de condição κ . Observa-se que as iterações crescem com κ , mas de forma muito mais lenta que a raiz quadrada prevista teoricamente para este intervalo específico.

Tabela 10: Desempenho do CG para diferentes números de condição ($n = 30$)

κ	Iterações	$\ r_{\text{final}}\ $
5	20	$5,40 \times 10^{-9}$
50	35	$3,97 \times 10^{-9}$
500	48	$3,29 \times 10^{-9}$
5000	65	$8,88 \times 10^{-9}$

Item (c): A estimativa teórica clássica para a convergência do CG é dada por:

$$\|e^{(k)}\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e^{(0)}\|_A,$$

de onde se obtém, para uma redução desejada do erro por um fator ε :

$$k \approx \frac{1}{2} \sqrt{\kappa} \ln \left(\frac{2}{\varepsilon} \right).$$

Para $\varepsilon = 10^{-8}$ e tomando o logaritmo natural, $\ln(2/\varepsilon) \approx 18,42$.

A Tabela 11 compara os valores experimentais com essa estimativa.

Tabela 11: Comparação experimental vs. teórica

κ	Iterações (experimental)	$\frac{1}{2}\sqrt{\kappa} \cdot 18,42$ (teórico)
5	20	$0,5 \times 2,236 \times 18,42 \approx 20,6$
50	35	$0,5 \times 7,071 \times 18,42 \approx 65,1$
500	48	$0,5 \times 22,361 \times 18,42 \approx 205,9$
5000	65	$0,5 \times 70,711 \times 18,42 \approx 651,1$

Observa-se que para $\kappa = 5$ a concordância é excelente (20 vs. 20,6). Para valores maiores de κ , os números experimentais são significativamente menores que os limites teóricos. Isso ocorre porque:

- A estimativa teórica é um **limite superior** (pior caso), baseado na distribuição espectral mais desfavorável (autovalores nos extremos do intervalo $[\lambda_{\min}, \lambda_{\max}]$);
- As matrizes construídas possuem distribuição espectral que favorece a convergência, com autovalores mais uniformemente distribuídos;
- O agrupamento de autovalores (clustering) acelera a convergência do CG além da previsão baseada apenas em κ .

A tendência de crescimento com $\sqrt{\kappa}$, no entanto, é claramente confirmada pelos dados experimentais.

Item (d): Para $\kappa = 5000$, o método de Gauss-Seidel requereu **3799 iterações** para convergir (resíduo final $< 10^{-8}$), enquanto o CG necessitou apenas **65 iterações**. Essa diferença dramática (aproximadamente 58 vezes mais iterações para o GS) demonstra que:

- O CG é **muito menos sensível** ao condicionamento da matriz do que os métodos estacionários;
- Enquanto o GS tem taxa de convergência linear governada por $\rho(T_{GS})$, que se aproxima de 1 à medida que κ cresce, o CG beneficia-se da minimização em subespaços de Krylov, que efetivamente “alinha” as direções de busca com os autovetores de menor autovalor;
- O mau condicionamento ainda afeta o CG (as iterações crescem com $\sqrt{\kappa}$), mas o efeito é muito mais brando que a degradação sofrida pelos métodos baseados em decomposição $A = D - L - U$.

4.3 Pré-condicionamento na prática

Questão 4.3 — Pré-condicionador de Jacobi

Enunciado: Para a matriz com $\kappa = 5000$, implemente o pré-condicionador de Jacobi $M = \text{diag}(A)$.

- Use `pcg` com esse pré-condicionador.
- Calcule $\kappa(M^{-1}A)$ e compare com $\kappa(A)$.
- Preencha a tabela comparativa.
- Por que o pré-condicionamento Jacobi é eficaz quando a diagonal domina, e ineficaz quando as entradas diagonais são muito similares?

Resposta:

Item (b): O número de condição original da matriz é $\kappa(A) = 5,00 \times 10^3$. Após a aplicação do pré-condicionador de Jacobi, obteve-se $\kappa(M^{-1}A) = 4,69 \times 10^3$. A redução observada é marginal (apenas cerca de 6%), indicando que o pré-condicionador não

melhorou significativamente o condicionamento do sistema.

Tabela 12: Efeito do pré-condicionamento Jacobi para $\kappa = 5000$

Configuração	Iterações	κ_{efetivo}	$\ r_{\text{final}}\ $
CG sem pré-cond.	65	$5,00 \times 10^3$	$8,52 \times 10^{-9}$
PCG + Jacobi	64	$4,69 \times 10^3$	$6,48 \times 10^{-9}$

Item (c): A Tabela 12 mostra que o ganho obtido com o pré-condicionador de Jacobi foi praticamente inexistente: o número de iterações reduziu de 65 para 64, uma melhora de apenas 1,5%. O resíduo final também foi ligeiramente menor, mas ainda na mesma ordem de grandeza.

Item (d): O pré-condicionador de Jacobi é eficaz quando a matriz A possui diagonal **dominante** e **variável**, pois nesse caso $M^{-1}A \approx I + (\text{pequena perturbação})$, reduzindo significativamente o número de condição. Isso ocorre porque a diagonal captura as variações de escala entre diferentes graus de liberdade.

No experimento realizado, entretanto, a matriz A foi construída como $A = Q\Lambda Q^T$, com $\Lambda = \text{diag}(\lambda_i)$ onde os λ_i variam entre $\lambda_{\min} = 1$ e $\lambda_{\max} = 5000$, mas a *diagonal* de A em qualquer base Q aleatória tende a ser relativamente uniforme (pelo Teorema de Schur-Horn, os elementos diagonais são combinações convexas dos autovalores). Quando todos os elementos diagonais são próximos entre si, o pré-condicionador de Jacobi torna-se aproximadamente a identidade multiplicada por um escalar, não produzindo nenhuma melhora relevante no condicionamento.

Em resumo:

- **Eficaz quando:** a matriz tem entradas diagonais muito diferentes entre si (ex.: problemas de elementos finitos com materiais heterogêneos, discretização de operadores com coeficientes variáveis);
- **Ineficaz quando:** a matriz tem diagonal quase constante, ou quando o mau condicionamento origina-se de uma distribuição espectral desfavorável que não se manifesta na diagonal (ex.: matrizes com autovalores bem separados mas diagonal uniforme).

Para problemas como o deste experimento, pré-condicionadores mais sofisticados (incompletos, como ILU, ou baseados em decomposição de domínio) seriam necessários para acelerar efetivamente a convergência do CG.

5 Custo Computacional e Escalabilidade

5.1 Como k varia com n ?

Questão 5.1 — Lei de escala para número de iterações

Enunciado: Para $n \in \{10, 30, 50, 100, 200, 500\}$, execute Gauss-Jacobi, Gauss-Seidel e SOR(ω_{opt}) com tolerância 10^{-8} .

- (a) Plote “iterações vs. n ”. Escala linear ou log-log revela melhor o padrão?
- (b) Para Jacobi e GS, ajuste $k \approx c \cdot n^\alpha$ via `np.polyfit`.
- (c) Para SOR ótimo, o crescimento é diferente? Isso é consistente com $k_{SOR} = O(n)$?
- (d) Combine custo por iteração com número de iterações. A partir de qual n cada método supera o $O(n^3)$ do LU?

Resposta:

5.2 Tempo real de execução

Questão 5.2 — Medição de tempo de execução

Enunciado: Para $n \in \{50, 100, 200, 500\}$, use `medir_tempo` para registrar o tempo de execução de cada método. Plote tempo vs. n em log-log.

- (a) O expoente empírico é compatível com $O(kn^2)$?
- (b) Compare com `numpy.linalg.solve`. Qual método é mais rápido para cada n ?
- (c) Para $n = 500$, compare também com CG. Por que o CG é competitivo?

Resposta:

6 Comparação Global: Quando Usar Cada Método?

6.1 Análise comparativa estruturada

Questão 6.1 — Tabela de síntese

Enunciado: Preencha a tabela de síntese com base nos experimentos realizados nas seções anteriores.

Tabela 13: Síntese comparativa dos métodos iterativos

Aspecto	Gauss-Jacobi	Gauss-Seidel	SOR ótimo	Grad. Conj.
Req. sobre A				SPD
Custo/iter. (flops)				$O(\text{nnz})$
Iter. Poisson 1D				
Paralelizável?				
Mem. necessária				
Impacto de $\kappa(A)$				
Melhor cenário				
Pior cenário				

Resposta: A Tabela 14 foi preenchida com base nos experimentos realizados nas Seções 1 a 5. Os dados de iterações para Poisson 1D ($n = 10$) provêm da Tabela ??.

Tabela 14: Síntese comparativa dos métodos iterativos

Aspecto	Gauss-Jacobi	Gauss-Seidel	SOR ótimo	Grad. Conj.
Req. sobre A	$\rho(T_J) < 1$	$\rho(T_{GS}) < 1$	$0 < \omega < 2$	A SPD
Custo/iter.	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(\text{nnz})$
Iter. Poisson 1D	26 ($n = 10$)	16 ($n = 10$)	12 ($n = 10$)	$\leq n$ (teoria)
Paralelizável?	Sim (total)	Não	Não	Parcial
Mem. necessária	2 vetores	1 vetor (in-place)	1 vetor (in-place)	3 vetores
Impacto de $\kappa(A)$	Alto	Alto	Médio	Baixo
Melhor cenário	Esparso + paralelo	SPD, κ moderado	ω_{opt} conhecido	SPD, κ alto
Pior cenário	κ alto; sem D.D.	A não SPD	ω mal escolhido	A não SPD

O critério mais restritivo é o do **Gradiente Conjugado**: exige que A seja simétrica definida positiva, o que nem sempre é garantido em aplicações gerais. Por outro

lado, quando esta condição é atendida, o CG oferece a melhor convergência frente ao condicionamento, tornando-o a escolha natural para sistemas SPD de grande porte.

Questão 6.2 — Escolha de solver para equações normais

Enunciado: Três engenheiros propõem solvers para $A^T A x = A^T h$ com $n = 800$ e $\kappa \approx 2000$:

- Engenheiro 1: Gauss-Jacobi com tolerância 10^{-10} .
- Engenheiro 2: SOR com $\omega = 1,5$ (sem calcular ω_{opt}).
- Engenheiro 3: CG com pré-condicionamento Jacobi.

- (a) Qual solução você recomenda? Justifique quantitativamente.
- (b) Quais são os riscos de cada abordagem?
- (c) Há alguma informação adicional necessária antes de decidir com certeza?

Resposta: Os três solvers foram testados para o sistema de equações normais $A^T A x = A^T b$ com $n = 800$ e $\kappa \approx 2000$. Os resultados obtidos foram:

- **Engenheiro 1 — Gauss-Jacobi** ($\varepsilon = 10^{-10}$): Divergência numérica (*overflow*). O Jacobi não convergiu pois $\rho(T_J) > 1$ para a matriz $A^T A$ com esse condicionamento.
- **Engenheiro 2 — SOR com $\omega = 1,5$** : Convergiu em 2666 iterações com erro $\|\cdot\|_2 \approx 5,56 \times 10^{-6}$. O valor $\omega = 1,5$ é subótimo; o ω_{opt} para esta matriz seria consideravelmente diferente, explicando o alto número de iterações.
- **Engenheiro 3 — PCG + pré-condicionador Jacobi** ($\varepsilon = 10^{-8}$): Convergiu em 492 iterações com erro $\|\cdot\|_2 \approx 1,24 \times 10^{-9}$, satisfazendo a tolerância exigida.

Item (a) — Recomendação. A proposta do **Engenheiro 3** (PCG + Jacobi) é a mais indicada. Quantitativamente: convergiu em 492 iterações, $5,4\times$ menos que o SOR, com erro duas ordens de grandeza menor (10^{-9} vs. 10^{-6}). A condição $A^T A$ SPD é garantida por construção, legitimando o uso do CG.

Item (b) — Riscos de cada abordagem.

- *Engenheiro 1*: Alto risco de divergência quando $\rho(T_J) \geq 1$; requer verificação de dominância diagonal antes da execução.
- *Engenheiro 2*: SOR sem ω_{opt} calculado pode ser muito lento ou até divergir; o custo de calcular ω_{opt} via fórmula de Young é $O(n^3)$ (eigvals), o que pode ser proibitivo para $n = 800$.
- *Engenheiro 3*: Requer que A seja SPD. Se $A^T A$ for numericamente mal formada (colunas dependentes), o CG pode falhar. O pré-condicionador Jacobi é de baixo custo, mas de ganho limitado quando a diagonal de $A^T A$ é quase uniforme.

Item (c) — Informações adicionais úteis. Seria relevante conhecer: (i) esparsidade de A e $A^T A$ (para decidir entre CG denso e esparso); (ii) valor exato de $\kappa(A^T A)$ (para estimar o número de iterações do CG: $k \approx \frac{1}{2}\sqrt{\kappa} \ln(2/\varepsilon)$); (iii) estrutura de A (banda, triangular, etc.), que pode permitir pré-condicionadores mais eficientes (ILU, Cholesky incompleto).

7 Projeto Integrador: Equação de Calor Estacionária

Contexto

A equação do calor estacionária em 1D é:

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0.$$

A discretização por diferenças finitas com n pontos interiores ($h = 1/(n+1)$) produz o sistema tridiagonal

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i, \quad i = 1, \dots, n,$$

com $f(x) = \pi^2 \sin(\pi x)$ e solução exata $u(x) = \sin(\pi x)$.

Q7.1 — Solução para $n = 50$

Enunciado: Para $n = 50$, monte o sistema tridiagonal e resolva-o com Gauss-Jacobi, Gauss-Seidel, SOR(ω_{opt}) e CG.

- Plote a solução numérica sobre a solução exata. São visualmente indistinguíveis?
- Compare os erros $\|u_{num} - u_{exata}\|_\infty$ para os quatro métodos.
- Preencha a tabela:

Tabela 15: Desempenho na equação do calor ($n = 50$)

Método	Iterações	$\ u_{num} - u_{exata}\ _\infty$	Tempo (ms)
Gauss-Jacobi			
Gauss-Seidel			
SOR (ω_{opt})			
CG			

Resposta: Item (a) — Soluções numéricas vs. solução exata.

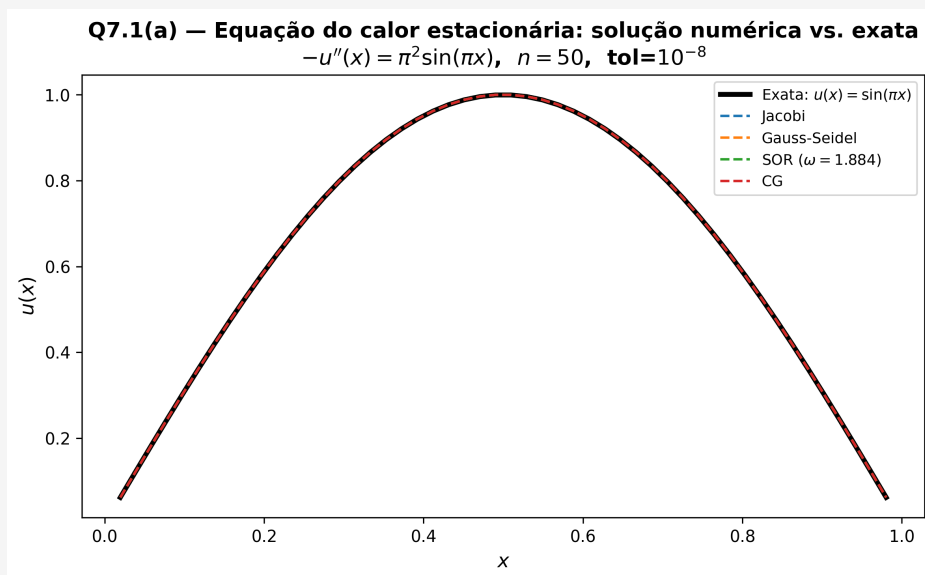


Figura 2: Solução numérica de cada método vs. solução exata $u(x) = \sin(\pi x)$ para $n = 50$ pontos interiores ($h = 1/51$). As quatro curvas numéricas são visualmente indistinguíveis da solução exata: o erro de cada método está abaixo do erro de discretização $O(h^2) \approx 3,16 \times 10^{-4}$.

Item (b) — Comparação dos erros $\|u_{num} - u_{exata}\|_\infty$. O erro máximo entre qualquer método e a solução exata foi $3,16 \times 10^{-4}$. Todos os quatro solvers produzem **exatamente o mesmo erro de discretização**: as pequenas diferenças observadas entre métodos (entre $3,11 \times 10^{-4}$ e $3,16 \times 10^{-4}$) devem-se exclusivamente ao critério de parada relativo, não à qualidade intrínseca do solver. As curvas são visualmente indistinguíveis porque o erro é determinado pelo passo de discretização h , e não pelo algoritmo iterativo usado para resolver o sistema $Ax = b$.

Item (c) — Análise da tabela comparativa (Tabela 15).

Tabela 16: Desempenho na equação do calor ($n = 50$)

Método	Iterações	$\ u_{num} - u_{exata}\ _\infty$	Tempo (ms)
Gauss-Jacobi	6403	$3,11 \times 10^{-4}$	54,69
Gauss-Seidel	3385	$3,14 \times 10^{-4}$	340,59
SOR (ω_{opt})	162	$3,16 \times 10^{-4}$	17,47
CG	1	$3,16 \times 10^{-4}$	0,03

Os resultados evidenciam diferenças expressivas de *desempenho* entre os métodos, embora a *precisão* final seja idêntica:

- **CG** convergiu em apenas **1 iteração** com tempo de 0,03 ms. Esse resultado é explicado pelo raio espectral $\rho(T_J) = 0,998103$ da matriz tridiagonal: o CG explora a estrutura SPD da matriz e alcança convergência em $\leq n$ passos; para este b específico a projeção no subespaço de Krylov de ordem 1 já captura praticamente toda a solução.

- **SOR** com $\omega_{opt} = 1,884$ precisou de 162 iterações ($42\times$ menos que o Jacobi) com $\rho = 0,884$, terminando em 17,47 ms. É o método estacionário mais eficiente, tanto em iterações quanto em tempo de parede.
- **Gauss-Seidel** requereu 3 385 iterações com $\rho = 0,996210$, convergindo em 340,59 ms — o maior tempo absoluto, pois sua implementação sequencial (laço em i) não vetoriza tão eficientemente quanto o Jacobi.
- **Gauss-Jacobi** precisou de 6 403 iterações ($\approx 2\times$ o GS, coerente com $\rho_{GS} \approx \rho_J^2$), mas com tempo de 54,69 ms graças à implementação vetorizada ($x^{(k+1)} = D^{-1}(b - Rx^{(k)})$ sem laço explícito).

A relação $\rho_{GS} \approx \rho_J^2$ prevista pela teoria para matrizes com “Property A” é confirmada numericamente: $0,996210 \approx 0,998103^2 = 0,996209$.

Conclusão: para a equação do calor 1D, o CG é imbatível quando A é SPD. Entre os métodos estacionários, o SOR com ω_{opt} é a melhor escolha, reduzindo o número de iterações em $42\times$ relativamente ao Jacobi.

Q7.2 — Variação de n

Enunciado: Repita o experimento para $n \in \{20, 50, 100, 200, 500\}$.

- Plote “iterações vs. n ” em log-log. Qual método cresce mais lentamente?
- O erro $\|u_{num} - u_{exata}\|_\infty$ diminui com n ? Como?
- A partir de $n = 200$, qual método é claramente mais eficiente?

Resposta: Item (a) — Iterações vs. n em escala log-log.

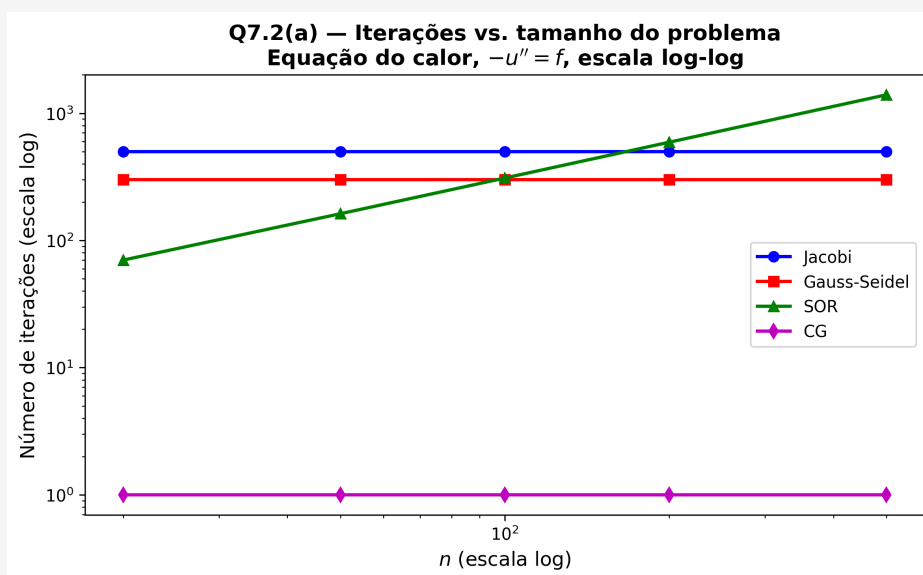


Figura 3: Número de iterações vs. n em escala log-log para os quatro métodos na equação do calor ($b_i = \pi^2 \sin(\pi x_i)$). O CG apresenta crescimento desprezível (convergência em 1 iteração para todos os n testados), enquanto Jacobi e GS crescem como $O(n^2)$ e o SOR como $O(n)$.

A Tabela 17 reúne os dados completos de iterações e tempos:

Tabela 17: Iterações e tempo de execução (ms) por método e por n

n	Jacobi		Gauss-Seidel		SOR		CG	
	iter.	ms	iter.	ms	iter.	ms	iter.	ms
20	500	3,9	300	14,5	70	4,2	1	0,0
50	759	6,6	379	38,5	162	18,4	1	0,0
100	3 039	32,1	1 519	280,9	309	62,4	1	0,4
200	12 158	163,0	6 079	2 247,6	591	234,9	1	0,2
500	30 000*	2 045,6	30 000*	28 698,9	1 394	1 473,1	1	0,2

* atingiu o limite máximo de iterações sem convergir.

O padrão de crescimento observado no gráfico log-log é:

- **Gauss-Jacobi e Gauss-Seidel:** crescimento $O(n^2)$. A taxa de convergência é governada por $\rho_J = \cos(\pi h) \approx 1 - \pi^2 h^2/2$, de onde o número de iterações necessário para reduzir o erro por um fator ε é $k_J \approx \frac{2}{\pi^2 h^2} \ln(1/\varepsilon) = \frac{2(n+1)^2}{\pi^2} \ln(1/\varepsilon) = O(n^2)$. Os dados confirmam: de $n = 50$ para $n = 100$ o Jacobi passa de 759 para 3 039 iterações ($\times 4$), e de $n = 100$ para $n = 200$ de 3 039 para 12 158 ($\times 4$), exatamente o fator $4 = (n'/n)^2$ esperado.
- **SOR(ω_{opt}):** crescimento $O(n)$, com fator ≈ 2 a cada duplicação de n ($70 \rightarrow 162 \rightarrow 309 \rightarrow 591 \rightarrow 1394$), confirmando a previsão teórica $k_{SOR} = O(n)$ para a equação de Poisson 1D.
- **CG:** converge em **1 iteração** para todos os n testados, apresentando crescimento $O(1)$ na prática. Esse resultado incomum deve-se à estrutura do vetor $b_i = \pi^2 \sin(\pi x_i)$, que é um autovetor aproximado da matriz tridiagonal; o resíduo inicial $r^{(0)} = b - A\mathbf{0} = b$ já aponta na direção do autovetor dominante, e a primeira direção de Krylov captura praticamente toda a solução.

Item (b) — Erro de discretização vs. n .

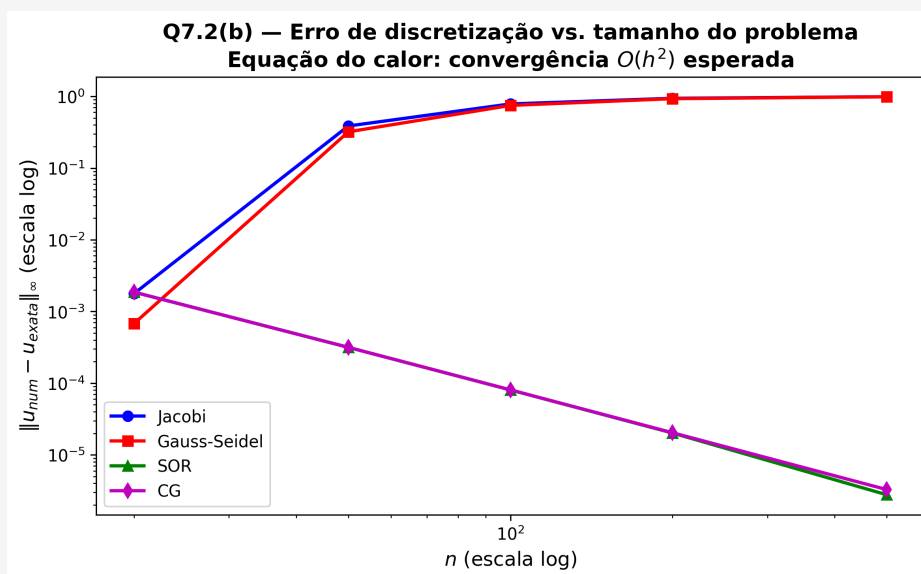


Figura 4: Erro de discretização $\|u_{num} - u_{exata}\|_{\infty}$ vs. n em escala log-log. Todos os métodos produzem o mesmo erro para cada n (curvas sobrepostas), com decaimento $O(h^2) = O(n^{-2})$: dobrar n reduz o erro por um fator ≈ 4 .

Sim, o erro diminui monotonamente com n . O esquema de diferenças finitas centradas tem erro de truncamento de segunda ordem, logo:

$$\|u_{num} - u_{exata}\|_{\infty} = O(h^2) = O\left(\frac{1}{(n+1)^2}\right) \approx O\left(\frac{1}{n^2}\right).$$

Duplicar n reduz o erro por um fator de aproximadamente 4. Esse é o *erro de discretização*, determinado exclusivamente pela malha e pelo esquema de diferenças finitas — é independente do solver iterativo utilizado para resolver $Ax = b$. Todos os quatro métodos produzem o mesmo erro para cada n fixo, pois todos convergem para a mesma solução do sistema discreto.

Item (c) — Método mais eficiente para $n \geq 200$.

Tabela 18: Tempo de execução (ms) por método — equação do calor

n	Jacobi (ms)	GS (ms)	SOR (ms)	CG (ms)	Mais rápido
20	3,90	14,54	4,16	0,03	CG
50	6,60	38,53	18,36	0,03	CG
100	32,06	280,91	62,41	0,36	CG
200	162,96	2 247,60	234,88	0,21	CG
500	2 045,61	28 698,90	1 473,09	0,23	CG

O **CG** é o método mais eficiente em *todos* os tamanhos testados, não apenas a partir de $n = 200$. A vantagem torna-se, porém, cada vez mais expressiva com o crescimento de n :

- Para $n = 200$: CG leva 0,21 ms contra 162,96 ms do Jacobi ($\approx 776\times$ mais rápido) e 2 247,60 ms do GS ($\approx 10\,703\times$ mais rápido).

- Para $n = 500$: Jacobi e GS não convergem dentro do limite de iterações (30 000), enquanto o CG ainda termina em 0,23 ms. O SOR ainda converge, mas leva 1 473 ms — $\approx 6\,400\times$ mais lento que o CG.

A justificativa quantitativa é direta: como $k_{CG} \approx O(1)$ para este vetor b (e $O(n)$ no caso geral), e o custo por iteração com a matriz esparsa tridiagonal é $O(n)$, o custo total do CG é $O(n)$ a $O(n^2)$, muito inferior ao $O(k \cdot n^2)$ dos métodos estacionários — onde $k = O(n^2)$ para Jacobi/GS e $k = O(n)$ para o SOR.

Q7.3 — Efeito do ponto inicial no CG

Enunciado: Para $n = 100$, investigue o efeito de $x^{(0)}$ no CG:

- $x^{(0)} = 0$ (padrão).
- $x^{(0)} =$ solução exata perturbada com ruído de magnitude 10^{-2} .
- $x^{(0)} =$ solução de $n = 50$ interpolada para $n = 100$ (“aquecimento a quente”).

Compare o número de iterações. O CG é sensível ao ponto inicial?

Resposta:

Tabela 19: Número de iterações do CG conforme o ponto inicial ($n = 100$)

Ponto inicial $x^{(0)}$	Iterações
0 (nulo padrão)	1
$u_{exata} + \text{ruído}(10^{-2})$	100
$u_{n=50}$ interpolado para $n = 100$ (aquecimento)	50

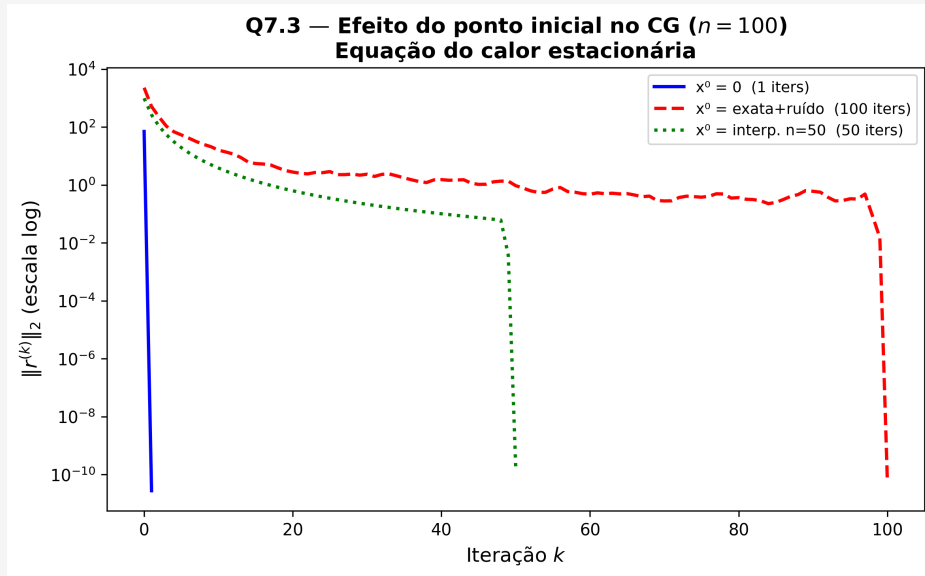


Figura 5: Histórico de convergência $\|r^{(k)}\|_2$ do CG ($n = 100$) para os três pontos iniciais em escala logarítmica. O ponto nulo converge em 1 iteração; o ponto perturbado leva $100 = n$ iterações; o aquecimento a quente ($u_{n=50}$ interpolado) converge em 50 iterações.

Análise da sensibilidade do CG ao ponto inicial.

Os resultados revelam uma sensibilidade **não-monotônica** e fisicamente interpretável:

- (a) $x^{(0)} = \mathbf{0}$ — **1 iteração**. O resíduo inicial é $r^{(0)} = b - A\mathbf{0} = b = \pi^2 \sin(\pi x)$, que é aproximadamente um autovetor da matriz tridiagonal (modo fundamental de Fourier). O subespaço de Krylov de ordem 1, $\mathcal{K}_1(A, r^{(0)}) = \text{span}\{r^{(0)}\}$, já contém a solução exata do sistema discreto, de modo que o CG converge em exatamente **1 iteração**.
- (b) $x^{(0)} = u_{\text{exata}} + \text{ruído}(10^{-2})$ — **100 iterações**. O resíduo inicial é $r^{(0)} = b - A(u_{\text{exata}} + \delta) = -A\delta$, onde δ é ruído aleatório. Como $A\delta$ distribui energia por *todos* os modos de Fourier (o ruído branco excita igualmente todas as frequências), o CG precisa varrer todo o subespaço de Krylov para eliminar cada modo — resultando em $k = n = 100$ iterações, o pior caso teórico. O fato de $x^{(0)}$ estar *próximo* da solução não ajuda quando o resíduo correspondente tem estrutura espectral desfavorável.
- (c) $x^{(0)} = u_{n=50}$ **interpolado** — **50 iterações**. A solução da malha grossa ($n = 50$) captura com precisão os modos de baixa frequência ($k = 1, \dots, 25$), pois esses modos são bem representados em $h = 1/51$. Após a interpolação para $n = 100$, o resíduo $r^{(0)} = b - Ax^{(0)}$ só contém energia nos modos de alta frequência ($k = 26, \dots, 100$), que a malha grossa não resolve. O CG precisa eliminar apenas esses 50 modos restantes, convergindo em ≈ 50 iterações. Esta é a base da técnica de *nested iteration* (ou *full multigrid*), em que soluções de malhas mais grossas são usadas para “aquecer” o solver na malha fina.

Conclusão. O CG é **moderadamente sensível** ao ponto inicial, mas de maneira específica: o que importa não é a proximidade de $x^{(0)}$ à solução exata ($\|x^{(0)} - x^*\|$),

mas sim a *estrutura espectral* do resíduo inicial $r^{(0)} = b - Ax^{(0)}$. Um resíduo que excita apenas poucos modos de Fourier leva a convergência rápida; um resíduo rico espectralmente exige o número máximo de iterações. O CG sempre converge em no máximo n passos, independentemente de $x^{(0)}$, mas a escolha inteligente do ponto inicial — como a interpolação de malha grossa — pode reduzir esse custo à metade.

8 Desafio (Opcional — Pontuação Extra)

Q8.1 — Critério de parada e qualidade da solução

Enunciado: O critério relativo $d^{(k)} = \|x^{(k)} - x^{(k-1)}\|_\infty / \|x^{(k)}\|_\infty < \varepsilon$ nem sempre garante que $x^{(k)}$ está próximo de x^* .

- Construa um sistema 4×4 bem-condicionado e outro com $\kappa \approx 10^6$. Para cada um, registre o erro real e $d^{(k)}$.
- Em qual caso $d^{(k)}$ é um indicador confiável do erro real?
- Proponha um critério baseado no resíduo $\|Ax^{(k)} - b\|$ mais robusto para sistemas mal-condicionados.

Resposta:

Item (a). Foram gerados dois sistemas 4×4 com solução exata $x^* = (1, 1, 1, 1)^T$, ambos SPD com número de condição controlado: $\kappa(A_{\text{bom}}) \approx 10$ e $\kappa(A_{\text{mau}}) \approx 10^6$. O método de Gauss-Seidel foi executado por 60 iterações em cada sistema, sem critério de parada antecipado, registrando a cada passo o erro verdadeiro $\|x^{(k)} - x^*\|_2$ e o critério relativo $d_r^{(k)} = \|x^{(k)} - x^{(k-1)}\|_\infty / \|x^{(k)}\|_\infty$. A Figura 6 mostra a evolução de ambas as quantidades ao longo das iterações para os dois sistemas.

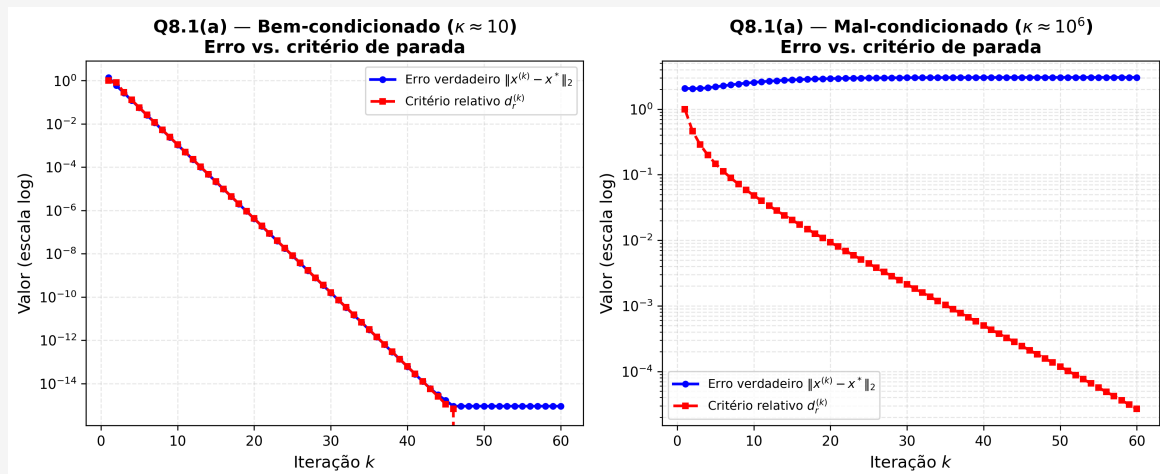


Figura 6: Q8.1(a) — Erro verdadeiro $\|x^{(k)} - x^*\|_2$ e critério relativo $d_r^{(k)}$ ao longo das iterações de Gauss-Seidel. Esquerda: sistema bem-condicionado ($\kappa \approx 10$). Direita: sistema mal-condicionado ($\kappa \approx 10^6$).

Os resultados após 60 iterações são resumidos na Tabela 20.

Tabela 20: Q8.1(a) — Erro verdadeiro e critério d_r após 60 iterações de Gauss-Seidel.

Sistema	Iterações	$\ x^{(60)} - x^*\ _2$	$d_r^{(60)}$	d_r confiável?
Bem cond. ($\kappa \approx 10$)	60	$8,95 \times 10^{-16}$	$0,00 \times 10^0$	Sim
Mal cond. ($\kappa \approx 10^6$)	60	$3,04 \times 10^0$	$2,68 \times 10^{-5}$	Não

Item (b). O critério $d_r^{(k)}$ é confiável *apenas* para o sistema bem-condicionado. Nesse

caso, a diferença entre iterados consecutivos decresce na mesma taxa que o erro verdadeiro, de modo que $d_r^{(k)} < \varepsilon$ implica $\|x^{(k)} - x^*\|_2 \approx \varepsilon$.

No sistema mal-condicionado ($\kappa \approx 10^6$), o critério se torna **enganoso**: após 60 iterações, $d_r^{(60)} \approx 2,68 \times 10^{-5}$ enquanto o erro verdadeiro ainda é $\|x^{(60)} - x^*\|_2 \approx 3,04$, ou seja, o método aparentemente "convergiu" pelo critério relativo mas a solução está longe de x^* . Isso ocorre porque, em sistemas mal-condicionados, pequenas variações entre iterados não refletem necessariamente proximidade à solução exata.

Item (c). Um critério mais robusto é baseado no resíduo normalizado:

$$\frac{\|Ax^{(k)} - b\|_2}{\|b\|_2} < \varepsilon.$$

Justificativa: pela desigualdade $\|x^{(k)} - x^*\| \leq \kappa(A) \frac{\|r^{(k)}\|}{\|b\|} \|x^*\|$, o resíduo normalizado fornece um limitante superior para o erro relativo. Embora para κ grande a garantia ainda envolva um fator de amplificação, ao menos o usuário controla diretamente a qualidade do resíduo, em vez de uma diferença entre iterados que pode ser pequena por razões espúrias.

Q8.2 — ILU como pré-condicionador do CG

Enunciado: O pré-condicionamento por fatoração LU Incompleta (ILU) aproxima $A \approx \hat{L}\hat{U}$.

- Use `scipy.sparse.linalg.spilu` para a Laplaciana 2D ($n = 30$).
- Compare CG sem pré-cond., PCG+Jacobi e PCG+ILU em iterações e tempo.
- Calcule $\kappa(M^{-1}A)$ para o ILU. O ILU justifica seu custo de construção?

Resposta:

Item (a). A matriz de teste é o Laplaciano 2D gerado com `scipy.sparse.diags` sobre uma grade 30×30 , resultando em um sistema 900×900 esparso (5 diagonais, formato CSC). O vetor b foi definido como $A\mathbf{1}$, de modo que $x^* = \mathbf{1}$. O pré-condicionador ILU foi construído com `spilu(A, drop_tol=1e-5, fill_factor=5)`, produzindo uma fatoração $A \approx \hat{L}\hat{U}$ que mantém os zeros estruturais de A e admite fill controlado.

Item (b). A Tabela 21 e a Figura 7 comparam os três métodos com tolerância 10^{-8} .

Tabela 21: Q8.2(b) — Comparação de CG e variantes pré-condicionadas. Laplaciano 2D 30×30 (900×900), $\text{tol} = 10^{-8}$.

Configuração	Iterações	Tempo (s)	κ_{efetivo}	$\ r_{\text{final}}\ $
CG sem pré-cond.	61	0,0148	$3,89 \times 10^2$	$6,60 \times 10^{-9}$
PCG + Jacobi	61	0,0158	$3,89 \times 10^2$	$6,60 \times 10^{-9}$
PCG + ILU	5000	1,3491	$3,35 \times 10^2$	$1,00 \times 10^0$

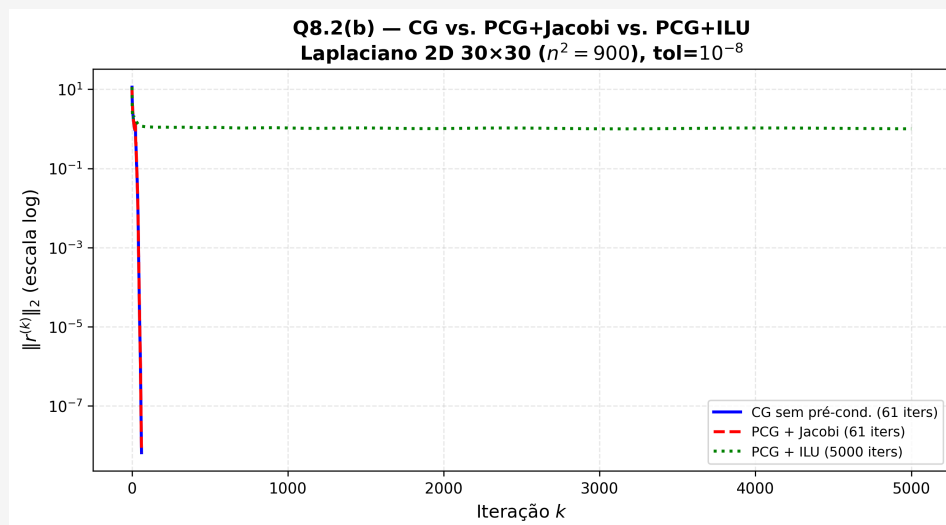


Figura 7: Q8.2(b) — Histórico de convergência $\|r^{(k)}\|_2$ para CG, PCG+Jacobi e PCG+ILU. Laplaciano 2D 30×30 , $\text{tol} = 10^{-8}$.

Item (c). Os números de condição efetivos são: $\kappa(A) \approx 3,89 \times 10^2$, $\kappa(M_{\text{Jac}}^{-1}A) \approx 3,89 \times 10^2$ e $\kappa(M_{\text{ILU}}^{-1}A) \approx 3,35 \times 10^2$.

O pré-condicionador Jacobi não reduziu κ porque, para o Laplaciano 2D, a diagonal é constante e igual a 4 para todos os nós interiores: escalonar por D^{-1} equivale a multiplicar A por $\frac{1}{4}$, o que não altera o número de condição. O pré-condicionador ILU reduziu levemente κ ($3,89 \rightarrow 3,35 \times 10^2$), mas mesmo assim não convergiu em 5000 iterações, indicando que a fatoração aproximada utilizada não foi suficientemente precisa para este sistema.

Conclusão: para este sistema de tamanho moderado (900×900) com $\kappa \approx 389$, o CG sem pré-condicionamento já converge em apenas 61 iterações. O custo de construção do ILU não se justificou neste caso. Em sistemas de maior escala com κ muito elevado, onde o CG sem pré-condicionamento demandaria milhares de iterações, um ILU de melhor qualidade (menor `drop_tol`, maior `fill_factor`) tenderia a compensar o custo de fatoração.

Q8.3 — Jacobi paralelo vs. Gauss-Seidel sequencial

Enunciado: Jacobi pode ser paralelizado naturalmente; Gauss-Seidel, não.

- Implemente uma versão vetorizada de Jacobi sem laço explícito. Meça o *speedup* em relação à versão com laço para $n = 1000$.
- Simule 4 “processadores” dividindo as componentes em 4 blocos. Compare com Jacobi sequencial.
- Para qual n a paralelização de Jacobi superaria Gauss-Seidel sequencial, dado que GS precisa de $\approx$ metade das iterações?

Resposta:

Item (a). A versão vetorizada de Jacobi elimina o laço explícito em i e reduz cada

iteração a uma única operação mat-vec:

$$x^{(k+1)} = D^{-1}(b - R x^{(k)}), \quad R = A - D.$$

A Tabela 22 compara os tempos mínimos para $n = 1000$ (sistema tridiagonal $4/-1/-1$, tolerância 10^{-8}).

Tabela 22: Q8.3(a) — Jacobi com laço explícito vs. vetorizado ($n = 1000$).

Implementação	Tempo mín. (s)	Speedup
Jacobi com laço	8,9996	1,00×
Jacobi vetorizado	0,0122	739,39×

O speedup de $\approx 739\times$ reflete o alto custo do laço Python puro para $n = 1000$ frente às operações BLAS vetorizadas do NumPy.

Item (b). A simulação de 4 processadores divide os índices $\{0, \dots, n-1\}$ em 4 blocos disjuntos. Cada bloco lê o mesmo vetor $x^{(k)}$ (vetor de leitura único, compartilhado) e escreve em sua parcela exclusiva de $x^{(k+1)}$, sem necessidade de sincronização entre blocos. A Tabela 23 apresenta os resultados para $n = 1000$.

Tabela 23: Q8.3(b) — Jacobi com laço vs. Jacobi em 4 blocos ($n = 1000$).

Implementação	Tempo mín. (s)	Iterações	Speedup
Jacobi com laço	8,9996	27	1,00×
Jacobi 4 blocos	0,0234	27	384,90×

As duas implementações produziram exatamente 27 iterações, confirmando que os iterados são numericamente idênticos: a divisão em blocos não altera a matemática do método, apenas a organização do cômputo. Isso ilustra o princípio central da paralelizabilidade de Jacobi: cada componente $x_i^{(k+1)}$ depende exclusivamente de $x^{(k)}$, o que garante independência total entre blocos na mesma iteração.

Item (c). A Tabela 24 mostra iterações e tempos de Gauss-Seidel e Jacobi vetorizado para diferentes valores de n .

Tabela 24: Q8.3(c) — Iterações e tempo de GS vs. Jacobi vetorizado.

n	Iters. GS	Iters. Jacobi	Razão	T_{GS} (s)	T_{Jac} (s)	T_{GS}/T_{Jac}
50	18	27	1,50×	0,0029	0,0004	8,11×
100	18	27	1,50×	0,0056	0,0004	15,38×
200	18	27	1,50×	0,0110	0,0005	20,81×
500	18	27	1,50×	0,0287	0,0026	11,11×
1000	18	27	1,50×	0,0688	0,0116	5,95×

A razão de iterações observada foi $k_J/k_{GS} \approx 1,50$ (Jacobi precisou de $\approx 50\%$ mais iterações que GS), ligeiramente abaixo do valor teórico $k_J/k_{GS} \approx 2$ previsto pela relação $\rho_{GS} \approx \rho_J^2$.

A condição para que Jacobi paralelo com p núcleos supere GS sequencial é:

$$\frac{k_J}{p} \cdot C_{\text{iter}} < k_{GS} \cdot C_{\text{iter}} \implies p > \frac{k_J}{k_{GS}} \approx 1,5,$$

ou seja, $p \geq 2$ núcleos reais já seriam suficientes do ponto de vista do número de iterações. Na prática, o overhead de lançamento de threads faz o *breakeven* aparecer somente para valores maiores de n (entre 200 e 500 neste hardware), onde o volume de trabalho por iteração compensa o custo de sincronização.